

IV. Proposed Method

A. Overview and Inputs and Outputs

GuardSynth preserves the action explanation in CoC while making conditions that restrict action selection explicit in a separate specification, then returning those conditions to a natural-language representation. Inputs comprise original CoC, scene observations available for the decision, and applicable rules with their evidence. Outputs comprise structured behavioral constraints, logical verification results under a restricted model, controlled natural language (CNL) generated from the specification, and an augmented learning input. The formal representation specifies condition semantics and verification scope; the language representation conveys those conditions to learning.

The process first selects candidate constraints and checks their evidence, then specifies accepted constraints in EBLC. Structural and semantic checks precede conversion to a verification-oriented intermediate representation and SMT queries. CNL corresponding to the specification is subsequently generated and combined with original CoC.

CoC + scene observations + rule evidence

- derive candidates and establish applicability conditions
- EBLC specification → Core lowering → SMT verification
- specification-based CNL generation → CoC augmentation

Observations, rules, and assumptions have distinct roles. Observations describe entities and events; rules determine restricted actions; assumptions delimit modeled situations. For example, zone clearance time is environmental information, the required subsequent waiting duration is rule information, and the absence of reoccupation is a modeling assumption. Different rules can therefore yield different admissible actions for the same observation.

Two executable profiles instantiate this approach. The primary learning experiment constructs numerical temporal contracts from supplied synthetic rules. A separate qualitative profile connects entities, zones, and events through observations and human review of real scenes. Both link specifications to evidence, but their inputs and semantics differ. We use the temporal contract as the running example and the qualitative profile to describe evidence uncertainty and constraint lifecycles. Implementation evidence is linked in [S1](#) and [S2](#).

B. Formal Specification with EBLC

1. Scope and Elements

A behavioral constraint restricts available actions in a situation. It differs from an observation, an action rationale, and an objective that ranks admissible actions. Satisfying an entry-permission condition does not force entry; selecting among permitted actions requires a separate objective.

Evidence-Bound Lifecycle Contracts (EBLC) represent constraints with their targets, conditions, action restrictions, persistence and release, and evidence. Evidence-Bound denotes explicit links to the inputs on which conditions and rules depend. Lifecycle denotes activation, persistence, release, and, where required, reactivation. This paper uses these organizing principles and implemented restricted profiles, rather than assuming a language covering every traffic situation.

Element	Specification role	Temporal instantiation
Subject and target	Whose action is restricted, and with respect to what?	Ego and pedestrian conflict zone
Scope	Situation and model of interpretation	Single clearance transition without reoccupation
Action restriction	What is disallowed?	Entry before release conditions hold
Persistence and release	When does the restriction persist or end?	Required duration following clearance
Time and units	Instants, durations, and boundaries	Integer milliseconds; inclusive threshold
Evidence and version	Rule sources and interpretation	Rule references and profile version

The qualitative profile additionally represents event truth, evidence validity, initial activation, and a policy for combining obligations. These elements address uncertain scene evidence; they are not all evaluated in the primary temporal learning experiment. The proposed modeling scope and experimentally evaluated scope are distinguished.

2. Temporal Syntax and Semantics

The temporal profile accepts a structured JSON contract. The following is actual generator output for a 0.5-second waiting rule. Source references provide traceability and contain no target action label.

```
{
  "version": "supplied-temporal-eblc-profile-v0.1",
  "subject": "ego",
  "target": "pedestrian_conflict_zone",
  "hold": "NO_ENTRY_WHILE_OCCUPIED",
  "release_clear_ms": 500,
  "time_unit": "ms",
  "source_kind": "SUPPLIED_SYNTHETIC_RULE",
  "assumption": "SINGLE_CLEAR_TRANSITION_NO_REOCCUPATION",
  "source_refs": [
    "Project01/temporal_guard_world.py::_guard",
    "paper1-method-preservation-v0.1"
  ]
}
```

The subject and target identify the actor and restricted zone. The hold field specifies no entry during occupancy; release_clear_ms specifies waiting after clearance. The assumption field specifies one occupied-to-clear transition without subsequent reoccupation. This permits continuous clearance to be represented by one clearance time and an elapsed duration.

Evaluation separately binds environmental clearance time clear_ms, candidate entry time entry_ms, and Boolean entry indicator enters. Both times use integer milliseconds. The restriction is:

$\text{enters} \rightarrow \text{entry_ms} \geq \text{clear_ms} + \text{release_clear_ms}$

For clear_ms = 11000 and release_clear_ms = 500, an entering candidate is compatible only at or after 11500ms. The inclusive comparison admits 11500ms but excludes 11499ms. A non-entering candidate has enters = false and does not violate this entry restriction; its progress is assessed separately.

The validator checks the supported version and fixed field combinations. Supported durations are 500, 1500, 800, and 1800ms; the primary expanded learning dataset uses 500 and 1500ms. The implementation accepts a restricted profile with defined interpretation, not arbitrary JSON as a general contract. [S1](#), [S3](#)

3. Qualitative Lifecycle Semantics

The real-scene profile uses `truth[j,t]`, `valid[j,t]`, and `active[j,t]` for obligation `j` at decision step `t`. Truth values are TRUE, FALSE, UNKNOWN, and CONFLICT. FALSE denotes confirmed resolution of the restricting condition, not mere absence of an observation. Validity denotes admissibility of the evidence. Activation records whether the obligation is active; its value before the first decision is a separate input.

```
on[j,t] = valid[j,t] AND (truth[j,t] = TRUE)
clear[j,t] = valid[j,t] AND (truth[j,t] = FALSE)
active[j,t] = TRUE, if on[j,t]
               FALSE, else if clear[j,t]
               previous_active, otherwise
```

Evidence is read and activation updated before constraining the action. Valid activation evidence activates an obligation, valid clearance evidence releases it, and otherwise its previous state persists. The same rule reactivates it when the activation condition is confirmed again. Explicit ordering and initialization fix the interpretation at each step.

The current policy permits entry only when clearance of every declared obligation is confirmed. Previous inactivity alone does not permit entry if current clearance is unconfirmed. When all obligations are clear, both entry and deferral are allowed. Permission is not a liveness guarantee requiring progress. UNKNOWN and CONFLICT are evidence states, distinct from an SMT solver returning UNKNOWN when it cannot decide a query. [S2](#)

C. Defining and Deriving Behavioral Constraints

1. Separating Candidate Constraints from Evidence

Not every CoC sentence becomes a constraint. “A pedestrian is crossing” is an observational claim; “yield to protect the pedestrian” is an action rationale. “Do not enter while the zone is occupied” contains a condition and restriction, making it a constraint candidate. “Choose the admissible candidate with greatest progress” is a selection objective. This distinction avoids inventing thresholds or release conditions absent from the explanation and its evidence.

Inputs serve three evidence roles. CoC identifies potentially relevant actions and obligations. Scene observations bind entities, zones, events, and times. Supplied rules or separately established normative evidence determine applicability and restriction content. A sentence appearing in CoC does not itself confirm applicability to the current scene.

2. Specification Procedure and Automation Scope

The procedure identifies action-relevant entities, situations, and rationales in CoC, then binds entities and zones to observations and identifies the decision time. Rule evidence

determines the restricted action and applicability conditions; persistence, release, timing, and assumptions are subsequently specified. Unconfirmed information remains unresolved rather than becoming an observed fact. Accepted content is mapped to supported fields with evidence references and version information.

In the primary synthetic experiment, the environment generator and supplied rule provide these inputs. Environmental clearance and rule duration determine the contract and verification representation. Automation structures and transforms given conditions; it does not discover traffic law or waiting durations from video alone. The real-scene path involves human confirmation of entities, zones, events, and applicability, and the implemented adapter is limited to a particular review task. [S3](#), [S4](#)

Constraints and learning references are managed separately. Defining admissibility does not automatically establish a real-scene ground-truth action. For the synthetic task, separate integer arithmetic uses known environmental states and rules to identify admissible candidates, then selects the one with greatest progress. This calculation does not copy its answer from CNL or an SMT verdict, but shares the environmental and rule assumptions.

3. Running Temporal Example

Original CoC reads “Yield to the crossing pedestrian, then proceed through the intersection.” The environment supplies clearance at 11.0 seconds; a separate rule requires a minimum wait of 0.5 or 1.5 seconds. The duration is an explicit experimental condition, not a value inferred from CoC or a universal traffic-law requirement.

Candidate	Entry time	Progress score	0.5-second wait	1.5-second wait
A	10.5s	110	Violates	Violates
B	11.5s	100	Admissible; reference	Violates
C	12.5s	85	Admissible	Admissible; reference
D	No entry	0	Admissible	Admissible

Progress is a synthetic utility, not measured distance or speed. B maximizes admissible progress under the 0.5-second contract and C under the 1.5-second contract. D violates neither but does not complete the progress objective. The table is a generated data instance, not evidence of actual model predictions. Candidate labels rotate during dataset construction to avoid linking the answer to a fixed letter.

D. Verifying EBLC Specifications Linked to CoC

1. Binding Checks and Core Lowering

Verification requires semantic input binding and logical queries. Subject, zone, event time, and duration must refer to the same case, and specifications must satisfy supported version, field, and unit requirements. Evidence identifiers support traceability rather than proving factual truth. Unbound fields are distinguished from completed scene bindings; passing syntax checks does not turn a conditional specification into reviewed learning data.

Temporal contracts lower to Core declarations of Boolean enters and integer clear_ms and entry_ms; release_clear_ms becomes an integer constant. Core is an intermediate representation of declarations and logical clauses, type-checked before SMT compilation. Time uses integer millisecond ticks with Core unit marker 1, not seconds. Variables in this profile are time-invariant and the clause is enforced initially. The configured horizon = 2 is a model bound, not verification of a two-second driving trajectory. [S1](#)

2. Properties and Queries

We distinguish consistency, restriction enforcement, release boundaries, and permissibility. Base SAT establishes a satisfying model, but non-entry alone can satisfy the specification. We therefore separately check that an entering model exists after release. This avoids specifications that exclude all entry without substituting for measurement of actual model progress.

Property	Query	Expected	Interpretation
Consistency	Base contract	SAT	A satisfying case exists
Restriction	Entry earlier than release	UNSAT	Early entry and contract are incompatible
Before boundary	Entry at 11.499s; wait 0.5s	UNSAT	Entry just before release is excluded
At boundary	Entry at 11.500s; wait 0.5s	SAT	Equality is permitted
After boundary	Entry at 11.501s; wait 0.5s	SAT	Entry after release is feasible
Deferral	enters is false	SAT	Entry is not forced

Concrete times assume clearance at 11.0 seconds. The restriction query searches for a counterexample with enters true and $\text{entry_ms} < \text{clear_ms} + \text{release_clear_ms}$. This is symbolic rather than fixing both time variables; boundary rows bind concrete values. Even the symbolic result concerns the encoded inequality and assumptions, not video interpretation or rule selection.

The six manuscript queries were executed for both 500ms and 1500ms contracts, yielding 12 matching verdicts. The existing bridge’s 100 queries were rerun separately. Neither set constitutes new learning-effect evidence. Records preserve each query, expected and observed verdicts, and actual SMT-LIB output. [S5](#)

3. SMT Representation and Interpretation

The following readable SMT-LIB example expresses the core 0.5-second restriction. It is an explanatory representation, not the entire compiler output, which is retained in separate query files.

```
(declare-const enters Bool)
(declare-const entry_ms Int)
(declare-const clear_ms Int)
(assert (= clear_ms 11000))
(assert (=> enters (>= entry_ms (+ clear_ms 500))))
(assert enters)
(push)
(assert (= entry_ms 11499))
```

```
(check-sat) ; unsat
(pop)
(push)
(assert (= entry_ms 11500))
(check-sat) ; sat
(pop)
```

The first UNSAT verdict indicates incompatibility of early entry with the specification, not a defective specification. The second SAT verdict indicates compatibility of the selected entry. Neither includes optimality or real-world safety. UNKNOWN or unexpected verdicts are not accepted as successful verification. Structural checks, scene binding, logical verification, and CNL correspondence serve different roles.

E. CNL Generation and CoC Augmentation

1. Generating Sentences from Verified Specifications

CNL is generated from checked fields through fixed sentence rules. The temporal renderer accepts structurally validated contracts and maps zone and duration to text. It does not itself invoke the solver; specification verification precedes rendering. Candidate labels, reference actions, and progress scores are not generator arguments. S1

Actual output for the 0.5-second contract is:

Do not enter the pedestrian conflict zone while it is occupied. Enter only after the zone has remained clear for at least 0.5 s.

The first sentence identifies the zone and occupancy restriction; the second expresses release and duration. “At least” corresponds to an inclusive boundary, and “only after” specifies a necessary condition rather than an instruction to enter. The conversion maps 500ms to 0.5s and, under the same rule, 1500ms to 1.5s. Correspondence is interpreted for supported durations and the single-clearance, no-reoccupation assumption.

2. Scope of Semantic Correspondence

Generation preserves target, negation, conditional direction, value, and unit. Omitting “at least” or replacing “enter only after” with a requirement to enter after clearance would change admissibility. Fixed field mappings make these relationships explicit and inspectable.

Not all metadata appears in model-facing sentences. Versions and source references remain traceability metadata; CNL conveys action-relevant conditions. Modeling assumptions are specified in the environment and task definition. The claim is correspondence of action restrictions within that context, not literal one-to-one rendering of every JSON field. Freely rewritten sentences or additional profiles require separate semantic review.

3. Combining CNL with Learning Inputs

Original CoC provides an action rationale, and CNL supplies additional behavioral conditions. They are combined without replacing CoC with a specification. The augmented example reads:

Yield to the crossing pedestrian, then proceed through the intersection. Do not enter the pedestrian conflict zone while it is occupied. Enter only after the zone has remained clear for at least 0.5 s.

The image and action candidates accompany this text, while a separate reference calculation supplies the target label. P0 receives original CoC, P1 receives CoC with the existing natural-language constraint, and P2 receives CoC with EBLC-derived CNL. P1 and P2 both add constraints; their primary comparator is CoC-only P0. Raw EBLC JSON and solver verdicts are not supplied as answer hints.

P1 and P2 receive constraints during both training and evaluation. This evaluates selection using explicit rules, not directly whether identical behavior persists without evaluation-time constraints. P0 lacks the waiting duration, so its inputs can be identical for paired contracts requiring different reference actions. This information difference is disclosed as part of the comparison.

Traceable records retain image references, CoC, contracts and evidence, CNL, candidates, reference actions, and splits, separating model-visible fields from answers and verification fields. This supports reconstruction of what was specified from which evidence, how it was expressed, and which action served as the learning target. Training budgets and metrics are presented in the experimental section.

Implementation and Example Evidence

These links provide reproducibility evidence for manuscript review, separate from external related-work citations. They will connect to code and data availability and supplementary material in the integrated manuscript.

- [S1 Temporal contracts, Core lowering, verification, and CNL generation](#)
- [S2 Qualitative contract parsing and lifecycle semantics](#)
- [S3 Learning inputs and independent reference calculation](#)
- [S4 Restricted real-scene adapter](#)
- [S5 Executed manuscript queries and results](#)
- [Two example learning-input records](#)